

VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science & Engineering

SCHOLARSHIP FINDER

A Practical Course Project Report for CSE1021: Introduction to Problem Solving

Student Name: Atharva Chandra Srivastava

Registration Number: 26BCE10065

Course Title: Introduction to Problem Solving (CSE1021)

Evaluation Track: Flipped Evaluation / Build Your Own Project

Submission Portal: VITyarthi

Academic Term • 2026

2. Introduction

Every academic year, dozens of central ministries, state welfare departments, and private foundations open up financial aid schemes for students. Even though funds are available, many deserving candidates miss out simply because the information is scattered across obscure portals. Each scholarship also comes with its own web of eligibility criteria—family income caps, minimum board exam percentages, social categories, and qualifying disciplines.

Scholarship Finder is a console-based Python project designed to take the guesswork out of discovering financial aid. Rather than manually reading through long notifications, a student enters basic profile details—household income, past exam percentage, social category, and stream of study. The application then evaluates these against a verified database of 20 Indian scholarship schemes (including the PM YASASVI, AICTE Pragati, and INSPIRE SHE initiatives) and returns every program for which the applicant qualifies.

3. Problem Statement

Locating scholarship opportunities manually is frustrating and inefficient for several clear reasons:

- **Multi-Variable Qualification Rules:** Scholarships enforce simultaneous restrictions. Missing a cutoff by 1% or being just above an income bracket leads to instant rejection, yet students often spend hours filling out forms before discovering this.
- **Scattered, Hard-to-Compare Circulars:** Details about grant amounts, awarding bodies, and specific quotas are spread across dozens of government notices that lack a unified format.
- **Runtime Vulnerability in Input Systems:** Terminal applications built for everyday users must tolerate unpredictable typing—such as entering commas in income, typing text into numeric prompts, or writing categories in lowercase—without abruptly crashing.

The goal of this project is to build a reliable, modular Python program that organizes scholarship rules into a structured database, cleans and validates user inputs, and accurately filters matching opportunities using clear conditional logic.

4. Functional Requirements

The system is organized into three core modules:

4.1 User Interaction & Validation Module (`main.py`)

- Presents an interactive menu with options to search scholarships, view the entire catalog, or exit.
- Captures the candidate's name (`s_name`), defaulting gracefully to `"Candidate"` if left blank.

- Guards family annual income (`inc`) with a validation loop that accepts only non-negative numeric inputs.
- Guards exam marks (`pct`) so users cannot proceed unless the value sits cleanly between 0 and 100.
- Displays lists of supported social categories (`categories`) and academic streams (`streams`) to guide user input.

4.2 Central Scheme Database Module (`schemes_db.py`)

- Maintains an in-memory list of 20 verified Indian scholarships (`SCHOLARSHIPS`).
- Stores seven attributes per record: scheme name, provider, award amount, income ceiling, minimum percentage cutoff, eligible categories, and qualifying streams.
- Exposes the `get_all_schemes()` helper to supply data to other files without modifying global state.

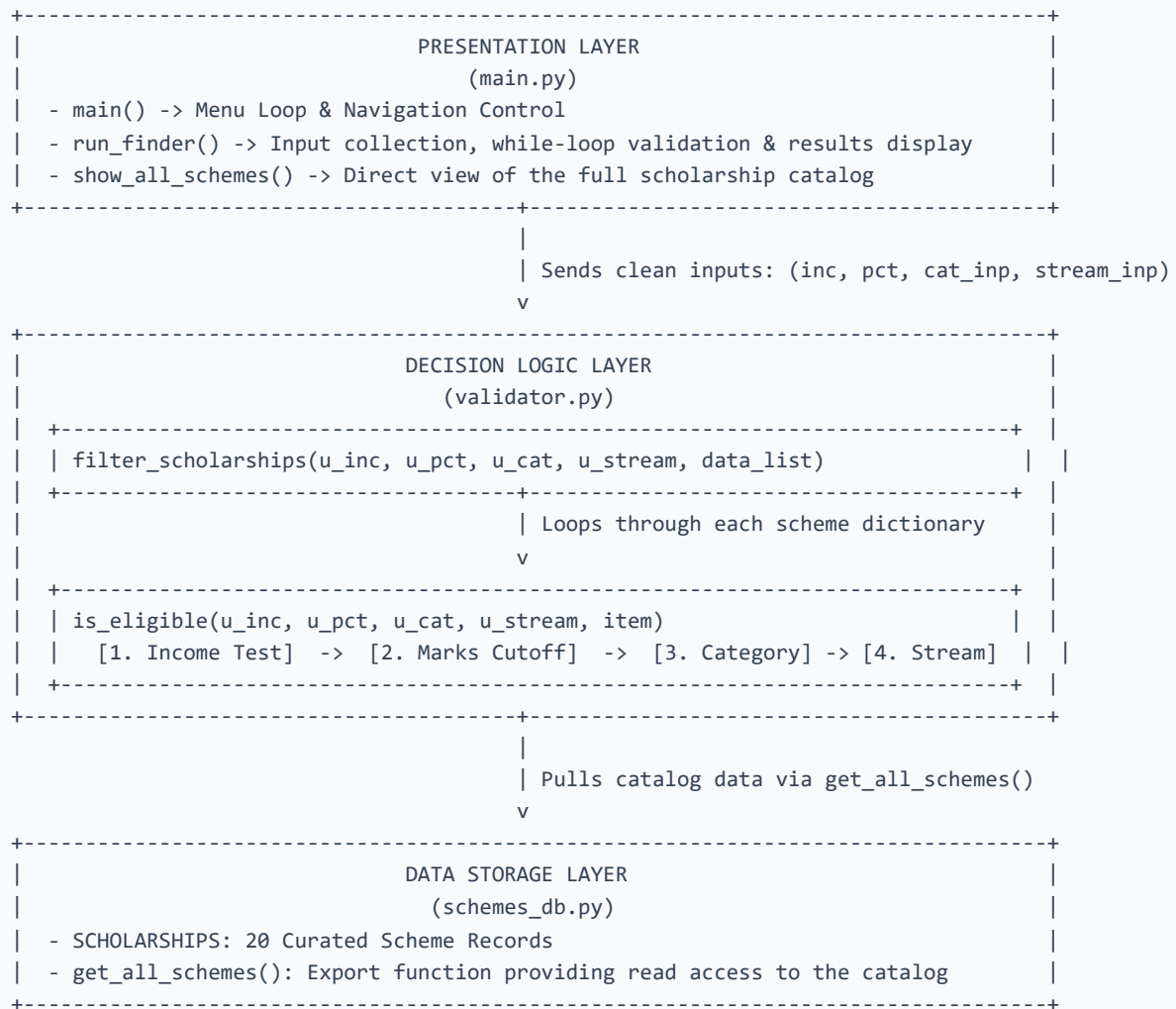
4.3 Decision & Filter Engine (`validator.py`)

- `is_eligible(u_inc, u_pct, u_cat, u_stream, item)` : Evaluates a candidate against an individual scheme record in four stages:
 1. Immediately rejects if applicant income exceeds the scheme's ceiling (`max_inc`).
 2. Immediately rejects if applicant marks fall below the required cutoff (`min_pct`).
 3. Performs a case-insensitive check against eligible categories, treating "All" as universal.
 4. Performs a case-insensitive check against qualifying streams, treating "All" as universal.
- `filter_scholarships(...)` : Runs each database scheme through `is_eligible` and returns the collected matches.

5. Non-Functional Requirements

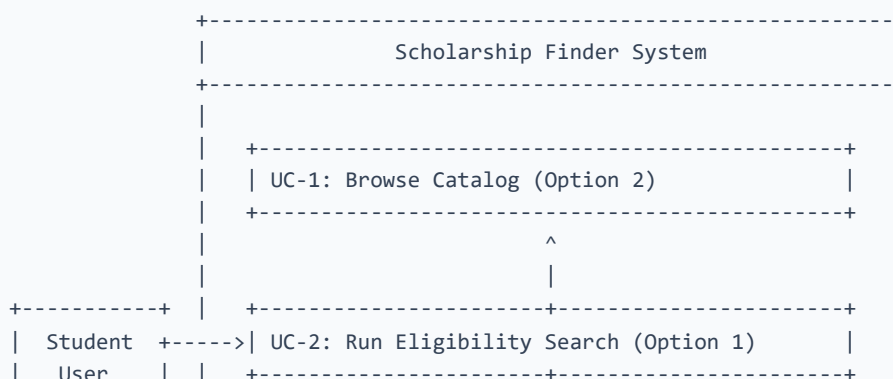
- **Responsiveness:** Traverses and filters the entire scholarship catalog in under 2 milliseconds on typical consumer PCs.
- **Usability:** Formatted cleanly with separators, clear numbered prompts, and readable card outputs that list scheme names, awarding agencies, and financial rewards.
- **Code Modularity:** Presentation logic (`main.py`), rule evaluation (`validator.py`), and data storage (`schemes_db.py`) remain separated across three files.
- **Defensive Robustness:** Wrapped in `try-except` blocks to safely handle non-numeric inputs without crashing or terminating unexpectedly.

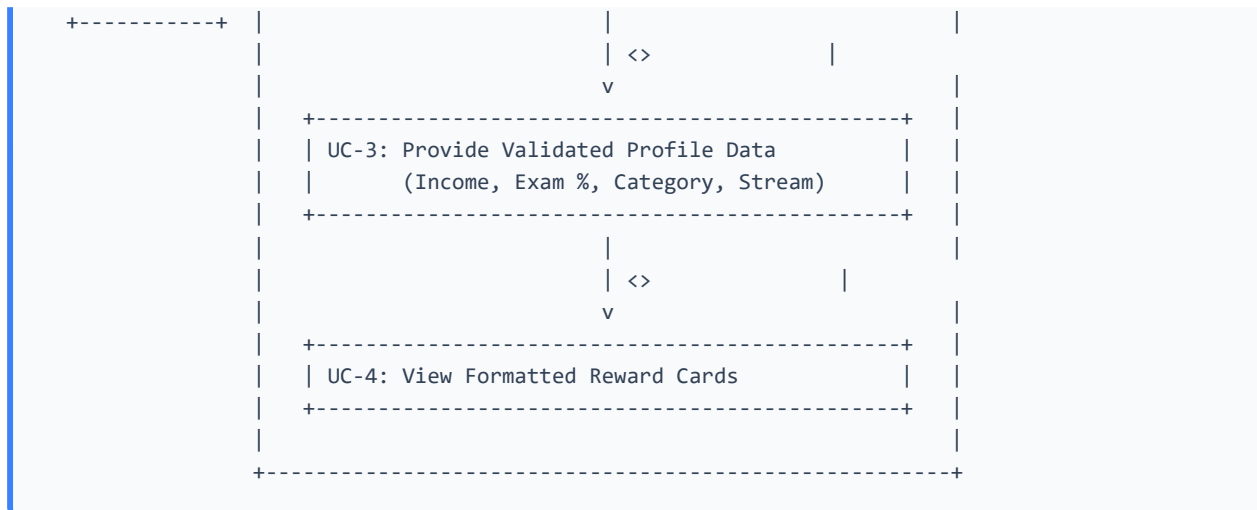
6. System Architecture



7. Design Diagrams

7.1 Use Case Diagram





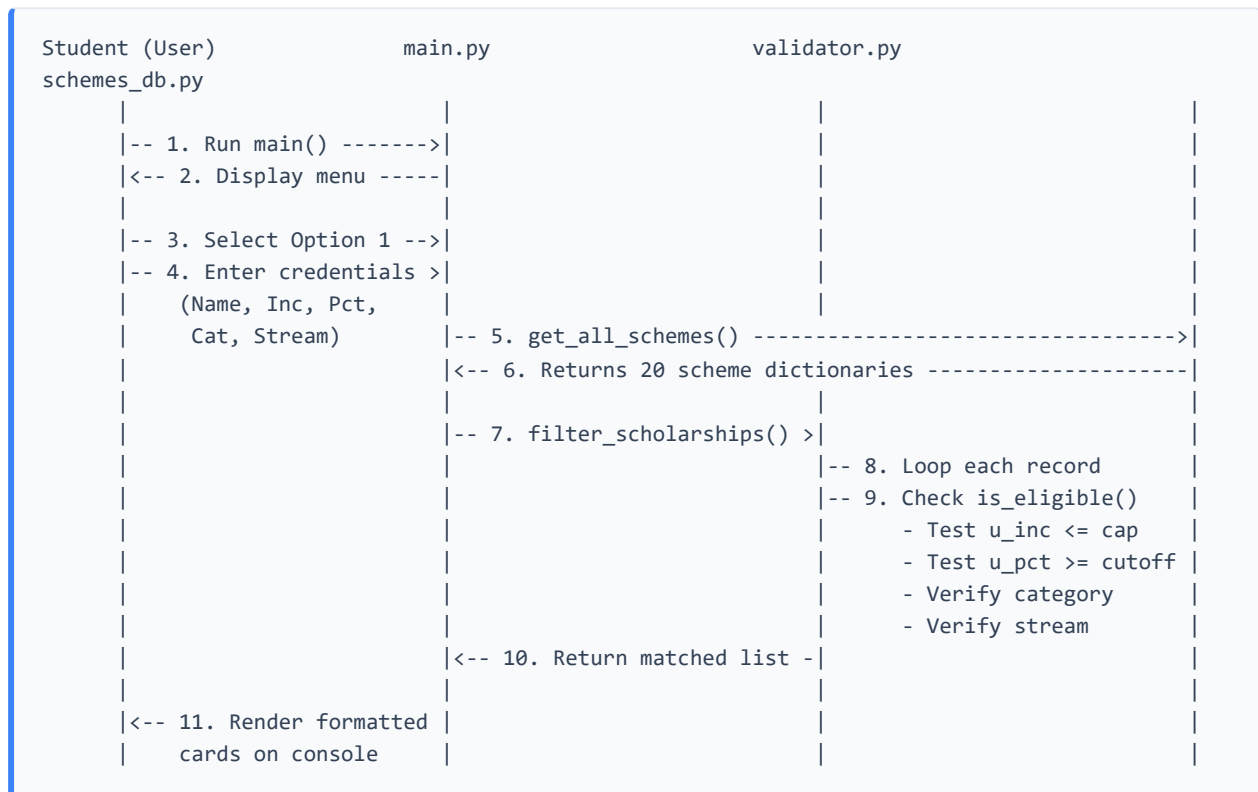
7.2 Workflow Diagram

```

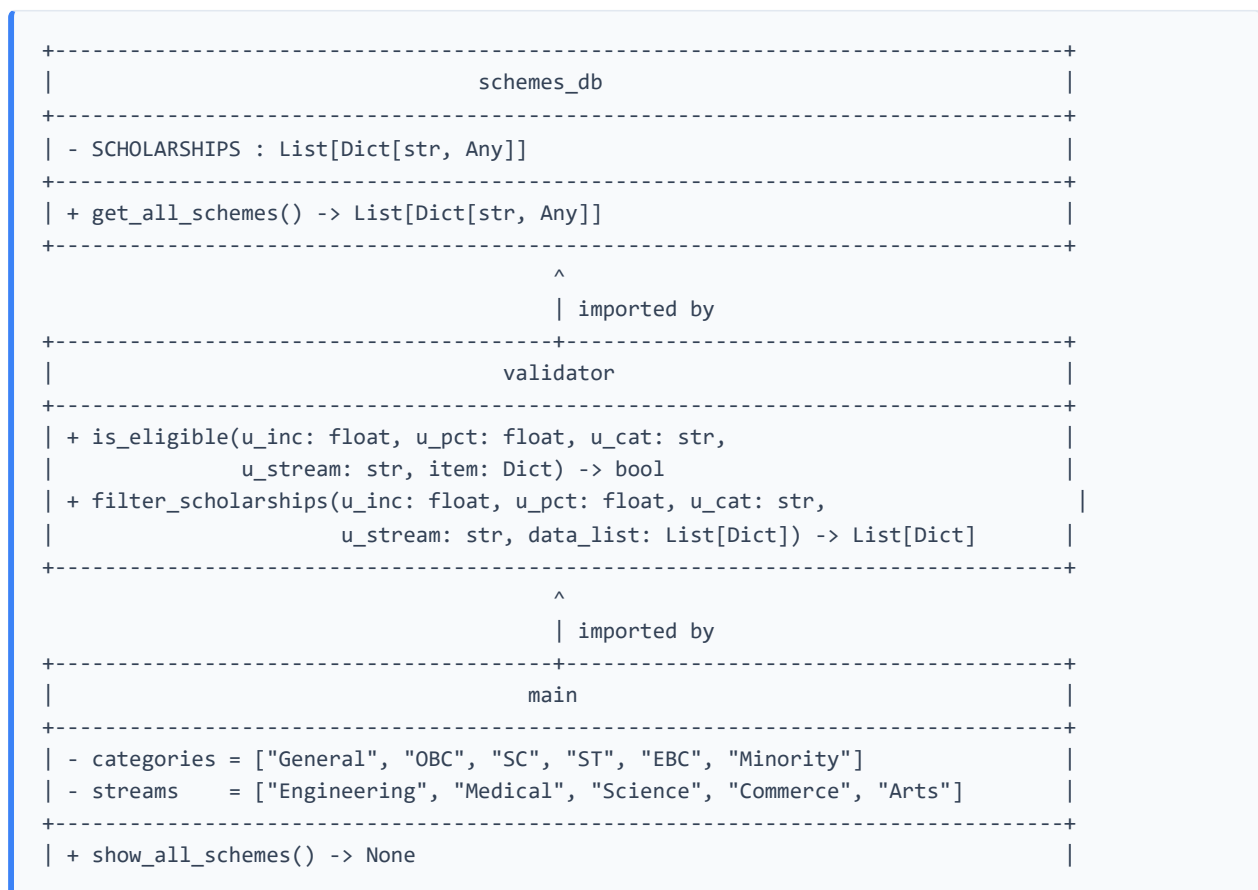
[Launch: main()]
|
v
[Display Main Menu (1: Find, 2: View All, 3: Exit)]
|
+----> [Option 2: show_all_schemes()] ---> [List All 20 Schemes] ---> [Loop back to Menu]
|
+----> [Option 3: Exit] -----> [Print Farewell] -----> [Terminate
Session]
|
+----> [Option 1: run_finder()]
|
v
[Prompt s_name: Default to 'Candidate' if left empty]
|
v
[While Loop: Prompt inc] ---> (inc < 0 or Non-numeric?) ---> [Show Warning]
| (Valid inc >= 0) ^
|
v
[While Loop: Prompt pct] ---> (pct < 0 or pct > 100?) ----->
| (Valid 0 <= pct <= 100)
|
v
[Prompt cat_inp from suggested categories]
|
v
[Prompt stream_inp from suggested streams]
|
v
[Fetch SCHOLARSHIPS from schemes_db.get_all_schemes()]
|
v
[Call validator.filter_scholarships()]
|
v
[For each scheme 'item' in catalog: is_eligible()]
|
+----> (u_inc > max_inc?) -----> [Discard Scheme]
+----> (u_pct < min_pct?) -----> [Discard Scheme]
+----> (u_cat doesn't match & not "All") -> [Discard Scheme]
  
```

```
reward]      +---> (u_stream doesn't match & not "All")-> [Discard Scheme]
              +---> (Passes all four conditions) -----> [Add to matched list]
              |
              v
[Check matched list count]
  |--(Empty)-----> [Print: "No matching scholarships found for this profile."]
  |--(Populated)-> [Print: Formatted scheme cards with name, provider &
                    |
                    v
[Return to Main Menu Loop]
```

7.3 Sequence Diagram



7.4 Component Diagram



```

| + run_finder() -> None
| + main() -> None
+-----+

```

7.5 Data Schema Diagram

CANDIDATE_INPUT				SCHOLARSHIP_RECORD			
PK	candidate_id	(INT)		PK	name	(VARCHAR)	
	s_name	(VARCHAR)	evaluated		provider	(VARCHAR)	
	inc (income)	(FLOAT)	against		amt (reward)	(VARCHAR)	
	pct (exam score)	(FLOAT)			max_inc	(INT)	
	cat_inp	(VARCHAR)	1	*	min_pct	(INT)	
	stream_inp	(VARCHAR)			cat	(LIST[STR])	
					stream	(LIST[STR])	

8. Design Decisions & Rationale

- **Clean Modular Separation:** Rather than writing everything inside a single script, splitting the project into `schemes_db.py`, `validator.py`, and `main.py` makes the codebase easier to read and maintain. Adding or updating scholarships never risks breaking the validation logic.
- **In-Memory Dictionaries Over Heavy Databases:** For a focused catalog of 20 to 100 schemes, an in-memory list of Python dictionaries provides instant lookups with zero installation or setup headaches for anyone evaluating the project.
- **Loop-Based Defensive Input:** Asking for numbers on the console frequently causes crashes if a user types words or enters negative numbers. Using `while True` loops wrapped around `try-except` blocks ensures the program catches mistakes gracefully and prompts the user again.
- **Ordered Short-Circuiting:** The checks in `is_eligible` evaluate numeric thresholds (income and exam percentage) first. Because arithmetic checks are faster than string matching, non-qualifying candidates are eliminated with minimal work.
- **Case Normalization:** Calling `.lower()` on both the user's input and scheme tags prevents accidental rejections caused by simple differences in capitalization (like typing "obc" instead of "OBC").

9. Implementation Details

The solution is implemented across three modular Python files:

9.1 schemes_db.py

Acts as the data store, containing a curated list of 20 schemes along with an accessor function:

```
# Sample schema structure from schemes_db.py
SCHOLARSHIPS = [
    {
        "name": "National Means-cum-Merit Scholarship",
        "provider": "Ministry of Education",
        "amt": "Rs. 12,000/year",
        "max_inc": 350000,
        "min_pct": 55,
        "cat": ["All"],
        "stream": ["All"]
    },
    ...
]

def get_all_schemes():
    return SCHOLARSHIPS
```

9.2 validator.py

Implements the core decision-making functions:

```
def is_eligible(u_inc, u_pct, u_cat, u_stream, item):
    if u_inc > item["max_inc"]:
        return False
    if u_pct < item["min_pct"]:
        return False

    cat_match = False
    for c in item["cat"]:
        if c.lower() == "all" or c.lower() == u_cat.lower():
            cat_match = True
            break
    if not cat_match:
        return False

    stream_match = False
    for s in item["stream"]:
        if s.lower() == "all" or s.lower() == u_stream.lower():
            stream_match = True
            break
    if not stream_match:
        return False

    return True

def filter_scholarships(u_inc, u_pct, u_cat, u_stream, data_list):
    matched_results = []
    for item in data_list:
        if is_eligible(u_inc, u_pct, u_cat, u_stream, item):
            matched_results.append(item)
    return matched_results
```

9.3 main.py

Manages the terminal interface, guides user input, and tabulates matches:

- `main()` runs the primary menu loop (options 1, 2, or 3).
- `run_finder()` gathers profile information with error handling, invokes the filter engine, and prints formatted scholarship cards.
- `show_all_schemes()` outputs the entire catalog for direct inspection.

10. Screenshots / Results

Scenario 1: Menu Navigation & Scheme Inspection (Option 2)

```
=====
      SCHOLARSHIP FINDER
=====
1. Find Eligible Scholarships
2. View All Database Schemes
3. Exit
Select an option (1-3): 2

Total Schemes Available: 20
-----
1. National Means-cum-Merit Scholarship
   Provider: Ministry of Education
   Max Income: Rs. 350000 | Cutoff: 55%
2. PM YASASVI Scholarship
   Provider: Ministry of Social Justice
   Max Income: Rs. 250000 | Cutoff: 60%
...
20. Aditya Birla Scholarship
    Provider: Aditya Birla Education Trust
    Max Income: Rs. 10000000 | Cutoff: 90%
```

Scenario 2: Profile Entry with Validation & Filter Output (Option 1)

```
Select an option (1-3): 1

--- Enter Candidate Profile ---
Enter Full Name: Atharva Chandra Srivastava
Family annual income (Rs): -25000
Income cannot be negative!
Family annual income (Rs): 220000
Last exam score (0-100%): 110
Score must be between 0 and 100.
Last exam score (0-100%): 78

Available Categories: General, OBC, SC, ST, EBC, Minority
Your Category: OBC
Available Streams: Engineering, Medical, Science, Commerce, Arts
Your Stream: Engineering

=====
Found 8 scholarships for Atharva Chandra Srivastava:
=====

1. National Means-cum-Merit Scholarship
   Provider : Ministry of Education
   Reward   : Rs. 12,000/year

2. PM YASASVI Scholarship
   Provider : Ministry of Social Justice
```

Reward : Rs. 75,000/year

3. AICTE Pragati Scholarship

Provider : AICTE

Reward : Rs. 50,000/year

4. Sitaram Jindal Foundation Scholarship

Provider : Sitaram Jindal Foundation

Reward : Rs. 500-1,500/month

5. Reliance Foundation UG Scholarship

Provider : Reliance Foundation

Reward : Up to Rs. 2,00,000

6. LIC Golden Jubilee Scholarship

Provider : LIC of India

Reward : Rs. 1,000-20,000/year

7. AICTE Saksham Scholarship

Provider : AICTE

Reward : Rs. 50,000/year

8. National Fellowship for OBC Students

Provider : Ministry of Social Justice

Reward : Rs. 31,000/month

11. Testing Approach

Test ID	Module	Test Condition / Input	Expected Output	Result
TC-01	main.py	Blank student name ("")	Assigns default fallback name "Candidate"	Passed
TC-02	main.py	Non-numeric income ("two lakhs")	Catches exception, shows "Please enter numeric characters only."	Passed
TC-03	main.py	Marks out of range (pct = -5 or 105)	Rejects input, prints "Score must be between 0 and 100."	Passed
TC-04	validator.py	Income exceeds cutoff (inc = 400000 for 350000 cap)	Evaluates to False and skips the scheme	Passed
TC-05	validator.py	Mixed case text (cat = "obc" , stream = "ENGINEERING")	Matches successfully against ["OBC"] and ["Engineering"]	Passed

Test ID	Module	Test Condition / Input	Expected Output	Result
TC-06	validator.py	Scheme marked with <code>cat</code> <code>= ["A11"]</code>	Qualifies applicants across all reservation categories	Passed

12. Challenges Faced

- **Uncontrolled Terminal Input:** During initial testing, mistyping a letter in the income prompt threw an unhandled `ValueError` and crashed the whole program. Wrapping the input conversion inside a `try-except` block within an infinite `while True` loop fixed this, keeping the prompt open until valid data is entered.
- **List vs. String Categorization:** Several schemes cater to multiple groups (like `["OBC", "EBC"]`) while others accept `["A11"]`. A simple `==` comparison failed on lists, so an inner loop with boolean match flags (`cat_match` and `stream_match`) was introduced to handle both specific groups and open categories reliably.

13. Learnings & Key Takeaways

- Gained practical experience breaking a project into logical files, keeping presentation separate from core logic and data storage.
- Learned how to write defensive terminal applications that anticipate user mistakes and handle them without crashing.
- Deepened understanding of core Python data structures—especially traversing nested lists and dictionaries with conditional checks.

14. Future Enhancements

- **External File Storage:** Move the schemes catalog from Python code into an external `schemes.json` or CSV file so updates can be made without touching source code.
- **Simple Web Interface:** Build an intuitive web UI using Streamlit or Flask so students can select choices from dropdown menus on their phones.
- **Exporting Results:** Add an option to save matching schemes directly into a clean `.txt` or PDF summary report.

15. References

1. Python Software Foundation. *Python 3.12 Documentation: Control Flow and Error Handling*.
<https://docs.python.org/3/>
2. National Scholarship Portal (NSP), Government of India. *Guidelines and Eligibility Criteria*.
<https://scholarships.gov.in/>
3. Vellore Institute of Technology. *CSE1021: Introduction to Problem Solving - Lecture Notes & Lab Exercises*.